COMP6000 Restaurant Booking System
Technical Report

lj330@kent.ac.uk , mz300@kent.ac.uk
oh219@kent.ac.uk , oj54@kent.ac.uk
sw734@kent.ac.uk

School of Computing
University of Kent

March 25, 2025

Abstract

This is the paper on our COMP6000 group project called the Restaurant Booking System. In this paper you will find all the information you will need for this project, it will include the technical reviews of the database, end of sprint testing outcomes and in-depth explanations on the technologies we used (to name a few).

This paper shall begin with an introduction to our project giving you the context and background needed to better understand the project. This will be followed by the background reading and evidence we have gathered throughout the project to reinforce the use case and relevance of this project and then go into the aims and the detailed explanations on used technologies and methodologies.

At the end of this paper, you will find the bibliography and appendices which support our statements and give reference to our background reading as well as our own materials.

We have enjoyed creating this project, so we hope you enjoy it to.

1 Introduction

The restaurant Booking System is a web application made for our restaurant La Dolce Vita, but ultimately it is a demonstration of what a restaurant's website could be when being developed with the newest frameworks and methodologies. We've not only created an interactive and dynamic website for a restaurant, we have also implemented an admin panel where authorised users have the ability to make additions and delete menu items, edit or delete reviews, handle questions sent from users and many more features which shall be explained in this paper.

Inspiration for this project came about from our shared interest in eating out as well as having the option to dine from home, and from experience we can say that choosing a place to eat from can be challenging. This is where our website aims to bridge the gap between availability and preference.

When deciding on a restaurant to eat from it can be a lengthy process, you either go with the norm or you try something new, this is where the trouble begins. This conflict has been defined as choice overload, "choice overload can sometimes have serious consequences" (Colin Camerer, cited in Velasco 2018). To summarize, this is an article by Velasco which explains why people cannot decide what to order for lunch, the phenomenon of choice overload can be related here in the same way. Velasco explains that "As the number of options increases, the potential reward increases, but then begins to level off due to diminishing returns" [Velasco, 2018], this shows that when you have many options the chance of finding something better than your original choice increases, but as you keep searching adding more options does not increase the reward . You tend to browse their websites and get to their menu pages. These tend to lack character as they usually are the splitting image of their physical counterparts. We aim to use technology to our advantage and change this dull perception, giving everything varying from high-definition images of the food, full nutritional information, item description and much more.

An imperative feature we decided we needed to implement is modularity, by this we mean the website can be updated with new menu items and reviews by the consumers or the administrator. This means that without any prior knowledge to programming you can edit menu items, delete menu items, create menu items, edit reviews, delete reviews and create reviews all from the website itself.

This goes to show that restaurant websites can be of high quality and we aim to demonstrate to other restaurants and developers how it can be done with our project.

2 Background

Before we began with the development of this project we done a fair chunk of background research, this allowed us to get a better understanding of the varying qualities of websites that we aim to improve upon. Our background research involved interacting with different restaurant websites, some of high quality and some of lower quality. We done this as it gave us a better understanding of the shortcomings of these sites and gave us an idea of what we can improve upon. Another aspect of research we undertook was reading through various articles reviewing the quality of websites. This was essential to us as this allows us to get an insider's thoughts of sorts into the pros and cons of existing websites, with some of them being for Michelin star restaurants.

A place where essential background research was needed was on the frameworks and technologies we planned on using, with these being React, Laravel, Inertia, SQL and PHP. The first 3 mentioned are completely new to us so getting to understand these was imperative to us because without it we potentially could of used the frameworks either in an incorrect way or an inefficient way. We are competently familiar on SQL and PHP but it never hurts to revisit, especially as we're using them in a foreign way.

We will begin with giving an example of a low-quality restaurant website as well as one of high quality, these examples and others can be found in more depth linked to our corpus, these were made for research purposes but fit nicely into the background discussion. There were quite a few interesting findings and points from this research which I shall touch upon shortly.

2.1 Low Level Restaurant Website

When researching existing restaurant websites, something to look out for was websites that were made of a lower quality. Just because these websites are of lower quality did not mean we had nothing to learn from them, in most cases they are still decent websites which fall short when compared to companies with higher budgets or understanding of e-commerce importance. An example is the website for The Old Weavers [1]. A few positive notes to make are that in general, this website is bug-free with all pages working as intended. Also, when you first load onto the web page, it looks professional, greeted with a high-resolution image of a dish, and at first glance, the navigation bar seems professionally made.

As you begin to look closer at this website the flaws become more visible, for example even at present day of writing this the Christmas Menu and Valentines Menu are still visible and accessible from the navigation bar. To me this suggests that they do not

[1] The Old Weavers Website: `https://www.weaversrestaurant.co.uk`

maintain their website regularly which could leave me to infer that their restaurant also reflects this (speaking from experience this is not the case). Another shortcoming with this site is the sluggish loading times of images, which suggests to me they did not experiment with various ways of loading images to determine the best use case.

These 2 mentioned shortcomings are just an example of the discussions and problems we outlined which in the long run assisted us in not making the same mistakes when it came to developing our website.

2.2 High Level Restaurant Website

In contrast, when researching restaurant websites there were many already existing examples of high quality. As we researched more, we concluded that the companies with a longer online presence and higher understanding of e-commerce tend to have the more advanced websites. Here is an example of one of these high-quality websites we interacted with, it being Wildwood [2]. This website used many new and upcoming features such as unique styling, a food menu with images and nutritional values for every dish and an in house-built food delivery site. These 3 features were all discussion points for the group as we deemed them all to be features that we should at least consider implementing, the custom food delivery feature would be great we just needed to be sure it would fit our use case without being out of place.

Although there were many great talking points about this website, as with any there are some downsides which we could learn from. A few of these were the bottom footer being visible on any page except the home page, a singular static review being visible on the website sitting on the table booking page and the most important in my opinion being the fact that only 1 menu (being the current seasonal menu) was integrated into the website properly. The other menus (being group booking menu and morning menu) were PDF's. These findings greatly benefited us as these may have been things that we potentially would of not considered which would of negatively impacted our project. After using our website, you can see that in most cases we have focused more on what other websites have done poorly rather than what they have done better.

So in the end, after the research into low- and high-level websites we can safely predict that the reason these websites are of lower level is because they tend to be for the single restaurant's where on the other hand the high level restaurants tend to be for restaurant chains. This links into website maturity which shall be talked about next.

[2]Wildwood Website: `https://wildwoodrestaurants.co.uk`

2.3 Comparison Of Michelin Star Restaurants

Here is an article on comparing Michelin star restaurant websites [Daries, 2018], I believe this to be one of the more interesting articles we read when researching about this project with my reason for this being summarized in this quote... "Results show that website maturity and content development are positively related, and that the aforementioned restaurants are not taking advantage of the opportunities that the Internet offers, and show different progress depending on the country where they are located and the category." These findings suggest that as a website improves over time so does the quality of content created for it, but websites still do not fully utilize the benefits of the internet. Ultimately, this means that restaurant websites need to be more aware of covering all aspects, including online reservations, online ordering, website reviews and website modularity (these all being niche features of sorts which were all discussed by us to be decide if we should implement these or not).

One interesting point in this article is that "If we focus on restaurants we can say that it is one of the sectors that has been most affected by Internet innovation" (Miranda Et Al 2015, cited in Daries 2018) . This point further enforces the fact that as web developers, we have an expectation for this project to go above and beyond existing examples which we can achieve through features such as online reservations, online ordering, social media marketing, digital menus and intuitive web pages. In contrast, innovation doesn't necessarily have to fall upon the content on the website. "Information search plays a very important role in consumer restaurant selection and decision-making" (Yilmaz and Gultekin 2016, cited in Daries 2018). My point is that another route for innovation could be to come up with new interesting ways to get onto the top of a search engine or other electronic searches, thus showing innovation can come in many ways.

So, to conclude my findings from this article it is seen that internet innovation has impacted restaurants far greater than other sectors, ultimately meaning we have to come up with fresh ideas as well as ensuring existing features go beyond what is already out in the wild. Innovation will lead us to success within this project.

2.4 Lavarel And React For Building Modern Web Applications

This short article describes the benefits of using these two technologies in tandem and gives some evidence to support the fact [Bhavsar, 2024]. It gives some context to what each individual framework achieves and the benefits from using them together, for example "Integrating these two will also benefit your web applications with high-quality performance, better development, feature-rich user experiences, and improved scalability." In this case, Laravel allows for high quality performance via its server-side logic and effective data caching while react ensures better development by allowing you to make

parts of your web application into components which allows you to reuse code without rewriting it.

The point in this article that resonated with me the most was "According to the StackOver Flow Report 2024, React and Laravel are among the top frameworks and technologies" [3]. Stack overflow is a community driven website that allows programmers to ask questions regarding their code and lets others come up with solutions in an attempt to help them, so the fact that React and Laravel are among the top web frameworks according to stack overflow shows promising results that we have chosen an effective combination of technologies. The last thing to take from this article was "React js with Laravel also offers secure features, such as SQL injection prevention, CSRF safety, and authentication". It was understood that if we deployed our project these would be concerns we would have to consider and tackle beforehand, but the fact that they have already been mitigated is the icing on the cake for using these frameworks.

3 Aims

These are the main outcomes we aimed to achieve with our project, spanning from the technical to theoretical, these aims partially came about from discussions from our weekly meetings with our supervisor as well as coming from our own weekly meeting discussions. Our background research assisted us in defining aims that made sense, aims that would support our ultimate goal of creating a web application that goes beyond existing high-quality examples.

In order, our main aims for the project were...

To implement a robust database written into a singular database script. There was a heavy emphasis throughout the project to keep updating the database script with any new tables and data that in the event of a failure needed to be added back to the database upon a refresh. We all had the understanding that this is a good practice as it meant that in catastrophic failure, we would have minimal loss.

Having a clear understanding and implementation of the Model View Controller architectural pattern. This was an absolute necessity as this was the fundamental pattern our frameworks expected for compilation, directory layout as well as logically.

Creating two intuitive layouts including the universal styling and web page structure. One for the core website and one for the admin panel. Achieving this meant that we would be efficiently using react as one of the fundamentals of using react is code reusability.

Having at least three fully implemented food menus, one for the morning, one for

[3]StackOver Flow Report 2024:
`https://survey.stackoverflow.co/2024/technology#3-web-frameworks-and-technologies`

evening and one for kids. This entails having images stored within the project rendered onto the page, including all relevant information on the specific food items being taken from the database. As seen in our background research, many restaurant websites would only include a single fully implemented menu with the other less important menus being PDF's so this was something we wanted to make sure to implement correctly.

Having dynamic components throughout the web pages. The term dynamic in the context of web development means that the data being rendered is coming from the database, allowing for updates on the web pages without necessarily updating the core code. This is one of the ways we aimed to achieve website modularity.

Including a review system on the website as well as a way for users to submit reviews with a chance of them being displayed on the website. This can be argued that this comes under the prior aim but as throughout the background research this would be one of our main gripes for restaurant websites, we believed this deserves to be a standalone aim.

Implementing a booking reservation system where users can choose a time and day. This entails an email confirmation system that emails the user with their booking confirmation, including their confirmation number, their day, their time and optionally a receipt of the food they have included in their order.

A food ordering system, entailing an option to choose which dishes the user wants, the quantity of dishes, a cart page where the user can edit their order and finally a checkout page where the user can confirm their order and pay. Although these payment methods include only dummy data and not actual card information, we felt the need for an authentic simulation of a payment process was needed to show that this project is deployment ready.

A full user account management suite. This included a login user page, an account registration page, an account information page and a change password page. This allows users to change any information on their account they need to. Again, this is an authentic implementation of this feature including all the latest hashing and authentication processes so if this project was fully deployed it would work in real-world scenarios.

And finally, the implementation of an admin panel. This panel includes an admin dashboard, a page rendering all reviews with the option to delete and edit existing reviews, a page rendering all the menus as well as their menu items with the option to delete or edit existing menu items, a page for adding new menu items, a page where admins can manage the incoming questions from users on a Frequently Asked Questions page and a full authentication system similar to users account feature. This is a large aim, but this will ensure modularity for our project and is something we have not seen implemented before.

4 Technical Analysis

Throughout this project, each group member has had the chance to learn about new technologies and use new frameworks, this has allowed each and every one of us to expand our knowledge on programming languages, methodologies and frameworks. Here is a short summary of how this project was made with in depth analysis of the separate components after.

To summarize, this project was created using the agile methodology, consisting of 2 meetings weekly, one with our supervisor and one amongst ourselves. The project was dissected into 9 sprints to allow for easy tracking of progress against time remaining. The backend logic is written in PHP using the backend framework Laravel, this gave us great libraries and even a starting point for our project using Laravel Breeze. Our frontend was programmed in JavaScript, using the frontend framework React which gave us a range of libraries, including ones to help gather user inputted information to be sent to the backend. Now, to get these 2 frameworks to interact with each other without any complications we used Inertia, this is a middleware that allows us to render the frontend pages within our Laravel routes as well as send data and requests to the backend via these routes. And lastly, we have a MySQL database hosted within the raptor environment, with the goal of allowing us to all to stay up to date and eliminate the chance of inconsistent data and tables which can hinder the project for group members, by using a single, centralised system we can avoid this.

4.1 The Database

As stated above, we decided to go with a MySQL database hosted on raptor. The reason we went with MySQL and not another SQL language such as Postgres was because we collectively had more experience with MySQL as well as the fact that Laravel defaults to this language. For each part of these technical analysis, we will share and explain some code snippets, to keep things consistent and easier to understand these shall be linked to the food menus, specifically here the morning menu. Here is an example of how we created a table in our database script.

```
CREATE TABLE morning_menu
(
    itemID INT AUTO_INCREMENT PRIMARY KEY,
    itemName VARCHAR(255) NOT NULL,
    itemPrice DECIMAL(4,2) NOT NULL,
    itemType ENUM('starter', 'main', 'dessert', 'extra') NOT
       NULL,
```

```
    -- ENUM is basically a varchar but it HAS to be one of
       these predefined strings
    itemDescription TEXT NOT NULL,
    itemImageURL VARCHAR(255) NOT NULL,
    -- URL is the directory where the image is stored, image
       on actual database too taxing
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
       CURRENT_TIMESTAMP
);
```

Listing 1: Creating the morning menu table

So, the interesting parts to note here are for starters how we handle the `itemPrice` field. We used a `DECIMAL(4,2)`, this means that the value of this field cannot surpass 99.99 because of it has to be 4 digits, 2 before the decimal point and 2 after in the form of `XX.XX` .

Next, we also use an interesting field called `itemType` using the `ENUM` type. This enumerator means that this field can only be one of the strings given to it as a parameter, in our case these are `starter`, `main`, `dessert` and `extra`. This helps with the backend validation of data.

Lastly, we have a field called `ItemImageURL`, there is no peculiar data type used here but rather the way it's used is interesting. The image directory route is stored in here rather than the image binary, this allows for images to be stored in the project rather than on the database in lower resolution.

4.2 The Model

For clarification, the model is used to establish a connection to the database by ensuring the table being targeted is the expected structure. Models are exact reflections of their corresponding tables and are responsible for data being created, retrieved, updated or deleted in a way that reflects the table's structure and upholds integrity. Here is the model used by morning menu.

```
class Menu extends Model
{
    // dynamically set the table name
    protected $table;

    // the rows to get from the database
```

```
    protected $fillable =
    [
        'itemName',
        'itemPrice',
        'itemType',
        'itemDescription',
        'nutritional_info',
        'itemImageURL',
    ];

    // ensures item price is float
    protected $casts =
    [
        'itemPrice' => 'float',
    ];

    // Dynamically setting the table name
    public function setTableName($tableName)
    {
        $this->table = $tableName;
    }
}
```

Listing 2: The universal menu model

Throughout the model, you can see the name space `protected` is used time and time again. The role of this is to allow the variables and properties using it to either allow access to it through this class or any other classes extending it but never from outside unconnected classes.

As you can see, this is a relatively simple model which is universal to every menu as it changes the targeted table on the fly via `$table`. This is achievable as each menu table is the same structure allowing for a simple table name change to work.

You can likely see similarities with the morning menu database table and the variable `$fillable`, this is because these are the fields found in that table which are fillable. This is how data can be sent and collected from the desired table.

If you look at the variable `$casts`, it ensures that the `itemPrice` is always of data type `float`. This ensures any prices taken from the database to be rendered on a page as well as prices being sent to the database as a new entry are always of float.

4.3 The View

To summarize, the view is the frontend which is rendered via the Laravel routes. These are the webpages which are rendered for the user in which they can interact with the website, it is also how information from the backend is rendered onto the webpages and is given a fancy looking user interface to allow the user to navigate the various web pages. The view also uses a great layout tailored for the user experience made by us. Here's a code snippet of the menu view.

```
import Layout from '../Layouts/GenericLayout';
import React, { useState, useEffect } from 'react';

const Menu = ({ menuType, menuTitle, menuDescription })
   => {
    const [menuItems, setMenuItems] = useState([]);
...

return menuItems.map((item) => (
    <div key={item.itemID} className="col-md-4 mb-3 d-
       flex justify-content-center">
        <div className="card" style={{ width: '18rem' }}>
            <img
                src={item.itemImageURL}
                classNamе="card-img-top img-fluid"
                alt={item.itemName}
                style={{ height: '200px', objectFit: '
                   contain' }}
            />
            <div className="card-body text-center">
                <h5 className="card-title">{item.itemName
                   }</h5>
                <p className="card-text">{item.
                   itemDescription}</p>
                <p className="card-text">
                    <strong>Price:</strong> £{item.
                       itemPrice.toFixed(2)}
                </p>
```

Listing 3: The universal menu view

To begin dissecting this code snippet, the `import Layout` at the top of the page is used to use the layout (which was mentioned above) which is the styling and structure of the web page, this allows for code reusability (we will explain layouts in more depth later). The `import React` is used to import various react libraries with `useState` being a hook which allows users to update fields to be sent to backend, often used in account creation but in this case used to track food items a user has chosen. The `useEffect` is a hook which is used to retrieve information from the backend to be rendered onto the web pages, in this case it's used to render the menu items onto the web page.

As seen, the `const Menu` is passed with 3 parameters, being `menuType`, `menuTitle` and `menuDescription`. These are in fact called properties (props for short) which are arguments passed to this page from a parent file which assists in rendering the correct information onto the page. In this case `menuType` tells the model which database table to target, `menuTitle` renders the title in the same way `menuDescription` renders description from said parent file.

The most interesting part of this code snippet is the `return menuItems.map((item)` segment, to summarize this is a react expression which loops through an array called `menuItems` which for each `item` in `menuItems` it adds a part to the array specific card component, for example it renders the image and then the item name, description and then price. Each array represents a single menu item and will loop through them all until all menu items are rendered onto the page.

Also within this code snippet, there is the use of Bootstrap which is a css framework which gives us some new and interesting ways of styling by giving us access to pre-configured classes. A few uses of this in this snippet are `card`, this adds the appropriate styling to make the menu items wrapped in a card, `card-img-top` ensures the image is at the top of the card and `img-fluid` ensures that the image corresponds to the user's screen size.

4.4 The Controller

The controller is basically the middleman between the database and the web page, it is how we get the required information stored on the database ready to be sent to the frontend. The database can be queried for data which is then sent to the frontend in the specified data type or structure such as JSON. The controller is not only accountable for taking data from the database but also the sending of data meaning that the controller essentially controls all data going in and out the database. Another feature of the controller is data validations, it can be used as a way to ensure data either being sent or retrieved is exactly what is expected, and if the data is invalid specific errors can be sent to the frontend to explicitly inform the user on what went wrong. Here is a snippet of

the menu controller.

```
// Method to submit a new menu item
public function submitMenuItem(Request $request)
{
    $validated = $request->validate([
        'name' => 'required|string|max:255',
        'price' => 'required|numeric|min:0|max:99.99',
        'type' => 'required|in:starter,main,dessert,extra',
        'description' => 'required|string',
        'category' => 'required|string|max:255',
        'menuType' => 'required|in:morning_menu,evening_menu,
           kids_menu',
        'image_url' => 'required|image|mimes:png|max:51200',
    ]);

    // Handle image upload
    if ($request->hasFile('image_url')) {
        $image = $request->file('image_url');

        $menuType = $validated['menuType'];
        $itemType = $validated['type'];
        $path = "menus/$menuType/$itemType";

        $storedPath = $image->store($path, 'public');
        $validated['image_url'] = 'storage/' . $storedPath;
    }

    $menu = new Menu();
    $menu->setTableName($validated['menuType']);

    // Create a new menu item
    $menu->create($validated);

    return redirect()->route('admin.manage-menu',
    ['menuType' => $validated['menuType']])->with('success',
       'Menu item added successfully.');
```

```
}
```

Listing 4: Creating the universal menu controller

This is the `submitMenu` function found in the menu controller, this is one of many controller functions found within this controller, but this function specifically allows admins from the admin dashboard to add new menu items to the specified menu. I chose this function as there are quite a few interesting points to be discussed.

Firstly, when this controller function is called it collects all the form information from the web page. This form is the data the admin has given to create a new menu item, this can be seen in the `$validated` variable which is the first part of the function. For example, `price` is a `required` field which is a float that has to be between `0` and `99.99` which reflects the menu table's `DECIMAL(4,2)` constraint. If the price collected from the web page does not fall in the range of `0` to `99.99`, an error message will be displayed underneath the price field on the web page and the controller function will return early and not go ahead.

Next, underneath the part just discussed you will see how we handle image uploads which are handled within this `if ($request->hasFile('image_url'))` statement. Again, the `image_url` field is `required` so if there is not an image handled by the controller in the same way explained above, there will be an error message under the image field stating either an image has to be uploaded or in the case of the image isn't of type `png`, the error message will state it needs to be of type `png`. If there is an image present, then the variables `$menuType` and `$itemType` are given their respective menu type and item type as this is going to be used to dynamically create the images file path which will be the `image_url`. Then `$path` is created using the two variables just discussed, this will be the main chunk of the file path, next `$storedPath` stores the image in the correct place using the path given and also gives it the `public` prefix which ensures it's stored in the storage directory. And finally, the `$validated[‘image_url"]` updates this field with the image's file path rather than the actual image as that is what the database expects as well as the image now being stored within the project.

Then once the image has been handled, the controller calls a new instance of the menu model, this is done by the `$menu` variable. This is how the data is structured in the way the database is expecting so it can be sent successfully. As the model is universal for all the menu tables, we needed to dynamically change which table it is targeting which is done by this line of code `$menu->setTableName($validated['menuType']);` . This makes sure that the table the admin has chosen in the form is the table the data will ultimately POST to.

And then finally the last few lines of this function, `$menu->create($validated);` creates the new table entry using all the validated data, this now means the new menu item is

in the database and can be seen within the website. Now the `return redirect()->route` line sends you to the webpage associated with admin.manage-menu route with a success message which renders on the web page in the same way the validation errors do.

4.5 Testing

As all well-designed websites should, we have gone through rigorous testing to ensure that our project makes sense and performs well in the hands of a user. From this testing it can inferred that no matter how many times you go through a feature yourself, (for example signing in and registering as a user) a test will always find a bug or feature that is not working as intended. We wanted to do our testing as follows, as we had 9 sprints we wanted to try and get testing done after each sprint. This was a hard goal to achieve due to deadlines and time so we did as many tests after sprints as we could manage. As a result, the testing is still thorough and cover a wide range of the project. Here's an example of how we tested features and theories.

```
render(<ChangePasswordPage/>);

    //Get inputs
    const currentPassInput = screen.getByLabelText(/current
       password/i);
    const newPassInput = screen.getByTestId('new-pass-input')
       ;
    const confirmPassInput = screen.getByLabelText(/confirm
       new password/i);
    const submitButton = screen.getByRole('button', {name: /
       register/i});

    //Fill out the form
    await userEvent.type(currentPassInput, 'OldPassword123');
    await userEvent.type(newPassInput, 'NewPassword123');
    await userEvent.type(confirmPassInput, 'NewPassword123');

    //Check input for correct values
    expect(currentPassInput).toHaveValue('OldPassword123!');
    expect(newPassInput).toHaveValue('NewPassword123!');
    expect(confirmPassInput).toHaveValue('NewPassword123!');

    //Simulate clicking register button
```

```
    await userEvent.click(submitButton);

    // Wait for the put function to be called with the
       correct data
    await waitFor(() => {
        expect(mockPut).toHaveBeenCalledWith('/password/
           change', {
            currentPassword: 'OldPassword123!',
            newPassword: 'NewPassword123!',
            newPassword_confirmation: 'NewPassword123!',
        });
    });
});
```

Listing 5: Unit testing for the change password page

This snippet is from one of our unit test files with this one specifically testing the change password page. The way this snippet works is that it renders the component via `render(<ChangePasswordPage/>);` , this will simulate the human interaction and behaviour. Next, all on screen elements are found with `screen.get` , these are the on-screen elements associated with input fields and buttons needed by this test to simulate changing the password. After this the test simulates a user inputting a password, which is seen here `await userEvent.type(currentPassInput, 'OldPassword123');` . This is 1 of 3 lines used to simulate a user typing a password and again is used to simulate the changing of a password. Now that this has been done the test now verifies the values it just inputting via the lines `expect(currentPassInput).toHaveValue` . And finally, the unit test simulates the button click then simulates a mock Api call to make sure the data is of expected result.

5 Conclusion

To conclude our project and this paper, collectively we feel that our website and its vision have proved that there is room for improvement in the restaurant website domain as well as e commerce in general. We have come up with original ideas, completed background research to prove that said features have not been implemented nor documented yet and improved on already existing ideas making our project sound (in our opinion).

5.1 Final Product quality

Overall, the quality of our project and the features which in fact make up this project are built to high quality. For example, we have gone through each file and made sure that we have comments which make sense and can be used by people in the future if the opportunity ever arose for someone (or us) to continue with this project.

All features that we implemented have been tested by us, we may have not implemented a unit test to explicitly show the functionality and validity of our features, but we can assure you that each feature was tested by us to ensure no bugs occur when in use.

5.2 Comparison To Similar Projects

When looking at similar projects taken by final year's, we can see some similarities with our project's final product. For example, after looking at other papers from individuals we can see that majority went for a web-based approach for the booking reservation system similar to how we done it. They also have their restaurant website included in the project similar to us.

One place where we differ on the other hand is that we used frameworks which are in fact used in the real world, these being React, Laravel and Inertia. Like projects went with the norm, JavaScript, HTML and CSS (and sometimes Bootstrap), this helps our project excel against others of the same scope as we were able to leverage unique features and styles by using the new (to us) frameworks.

5.3 Idea Novelty

With our project being already being attempted by many individuals and companies we had to get creative to achieve novelty in our project, with our most impressive feat of novelty being our admin dashboard. This allows our project to achieve modularity which we deemed as the most novel feature of our project as from our research nearly no one has done it or documented it before.

In fact, after researching in this scope to see if anyone has done it before we only found one well documented instance of this feature being published by the ShopurFood [4] team. They in fact went a lot more in depth with their features essentially making any part of the website maintainable and customizable from the admin dashboard. So ultimately, we believe that the fact that there is only 1 instance of this being done well leads us to believe that we did indeed achieve novelty with this project, not only just here but across the project.

[4]ShopurFood Admin Dashboard Article: `https://www.shopurfood.com/knowledge-base/admin`

5.4 Project Guidance

If we were to give some pointers to other groups or individuals who are just starting out on a similar project or have contrasting aims as us, the most valuable piece of advice we can give is spend time researching. Reflecting back on this project we understand that reading article after article can be daunting or just downright boring. But reading articles regularly on the various tasks or technologies you are intending to use will go a long way.

For example, a good way to start is to read about the broader task you are trying to achieve. Theres an interesting quote taken from an article about the essentials for any successful restaurant website, "When trying to set up a great restaurant website, it's easy to underestimate the value of a website that is easy to update. If it's too difficult to keep updated, the chances are good that it will always be a little bit behind." [Soares, 2024]. This is one of many great tips you will learn when reading ahead with my point being knowledge is power when it comes to being successful in this project.

5.5 Further Development

If we had more time or were to come back to this project, there are a few more tasks that could be done, including adding more modularity to the admin dashboard and building an in house food collection app for the drivers, the most prominent task we would want to do in the future is to make an application for restaurant tables in regards to the food and drinks ordering with the aim of reducing the amount of paper being thrown away to menus.

Theres a quite exciting quote we read on an article about switching from paper to tablets, "Digital menus, such as a tablet menu positively influenced the purchasing decisions of 91% of customers, resulting in a 38% increase in sales." [Cen, 2025]. This is a win win and would be an interesting feature to implement at a later date. There are quite a few promising figures to support this idea, "Displaying promotional banners on your ordering page can also help convince diners to buy the featured dish. A study by Nielsen found that sales of items advertised on digital signage increased by 33%." [Cen, 2025] The main point being that if we have more time for this project we would expand further into the restaurants with the aim of improving customer service further as well as potential revenue.

6 Acknowledgements

We would like to say a special thank you to our supervisor for this project, Dr Kemi Ademoye. Thank you for all the help throughout this project, being there every week for

our meetings as well as going beyond that to answer any questions that we may have. We couldn't have asked for a better supervisor.

Also, a congratulations to every group member. We all worked very hard and should be proud of the work we achieved; we worked well as a team, and it shows in the outcome.

Thank you for reading through our paper and we hope you enjoy our final product.

Bibliography

Dipal Bhavsar. How to use laravel with react for building modern web applications. *Technology Blog*, 2024. URL `https://www.bacancytechnology.com/blog/laravel-with-react`.

Junah Cen. Tablet menu: 5 reasons why you should do the switch. *Menu Tiger*, 2025. URL `https://www.menutiger.com/blog/tablet-menu`.

Natalia Daries. Maturity and development of high-quality restaurant websites: A comparison of michelin-starred restaurants in france, italy and spain. Technical report, University of Lleida, Lleida, Spain, 2018.

Soraia Soares. Essential elements and features of restaurant websites. *Boost*, 2024. URL `https://www.boostbrands.co.uk/insights/essential-elements-and-features-of-restaurant-websites`.

Emily Velasco. Scientists uncover why you can't decide what to order for lunch. *Caltech*, 2018. URL `https://www.caltech.edu/about/news/scientists-uncover-why-you-cant-decide-what-order-lunch-83881`.

A Appendix A: Entity Relationship Database Diagram

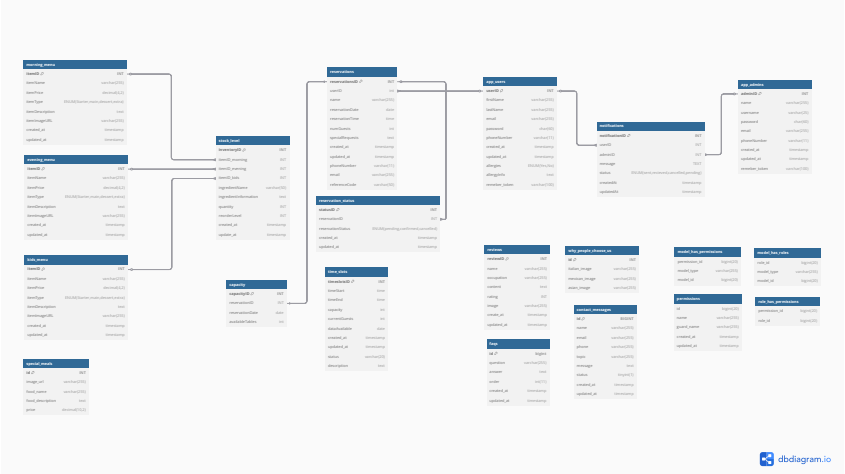

Figure 1: The entity relationship diagram for our database